Handwritten text recognition with Connectionist Temporal Classification (CTC)

Suggested reading: "Sequence Modeling With CTC" by Awni Hannun.

Text, audio, video, etc., are naturally modeled as sequences, and many problems involving them can be formulated as sequence-to-sequence processing.

Neural networks like RNNs and CNNs, can easily address some of the cases:

- many-to-one problems, where a single output is assigned to the whole input sequence;
- one-to-one problems, where there is one output element for each input element.

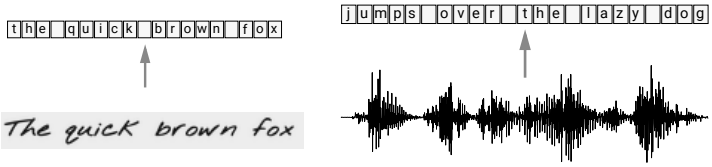
General many-to-many problems demand complex architectures involving multiple networks (e.g., encoder-decoder models).

Contextual Temporal Classification (CTC) is an efficient and elegant technique proposed in 2006 to deal with some many-to-many problems using a single RNN or CNN (read the original paper).

In particular, CTC addresses problems entailing segmentation and classification:

- the input sequence is segmented into groups of consecutive elements;
- a class label is assigned to each group;
- the output is the sequence of class labels.

Handwritten text recognition and speech recognition are examples of this kind of problems, where the characters are the class labels.



Handwritten text recognition

In this exercise we will use CTC to implement a system for handwritten text recognition.

We will used a processed version of the IAM Handwriting Database, including 5000 lines of handwritten text (4500 for training and 500 for test). For each line, a transcript with the correct text is available. Note that the lines do not form complete sentences, and may include English letters, digits, the punctuation symbols `,.'-!?:;"`, and the white space character.

Setup and Data

The dataset is distributed as an archive with the following structure:

```
handwritten/
├── train/
│   ├── images/
│   │   ├── 0001.png
│   │   ├── 0002.png
│   │   ├── 0003.png
│   │   ├── ...
│   │   └── 4500.png
│   └── sentences.txt
└── test/
    ├── images/
    │   ├── 4501.png
    │   ├── 4502.png
    │   ├── 4503.png
    │   ├── ...
    │   └── 5000.png
    └── sentences.txt
```

The two `sentences.txt` files report the text displayed in the images, one per line, in the order following the enumeration of the files. Images have been already resized to 400×32 pixels.

Let's start by importing all the libraries we will need and extracting the data from the archive. The archive also includes the `edit_distance.py` module, which we will also need later.

```
import torch
import torch.nn.functional as F
```

```
import torchvision
import matplotlib.pyplot as plt
import google.colab
import zipfile
import os

google.colab.drive.mount('/content/drive')
PROJECT_DIR = "/content/drive/MyDrive/Colab Notebooks/"  # Replace with the correct path
zip_ref = zipfile.ZipFile(PROJECT_DIR + "handwritten.zip", 'r')
zip_ref.extractall(".")
zip_ref.close()

from edit_distance import edit_distance
print("OK")
```

```
Mounted at /content/drive
OK
```

Dataset and data loaders

Now we need to setup the datasets and the data loaders. We can reuse the `ImageFolder` class provided by the `torchvision` module. However, we need to replace the useless class labels with the information read from the `sentences.txt` files.

It would be better if all sentences were the same length so that we could use a single tensor to represent a batch of text. We assume that no sentence is longer than 30 characters, and we will use a special `PAD` character to fill sentences that are shorter than that.

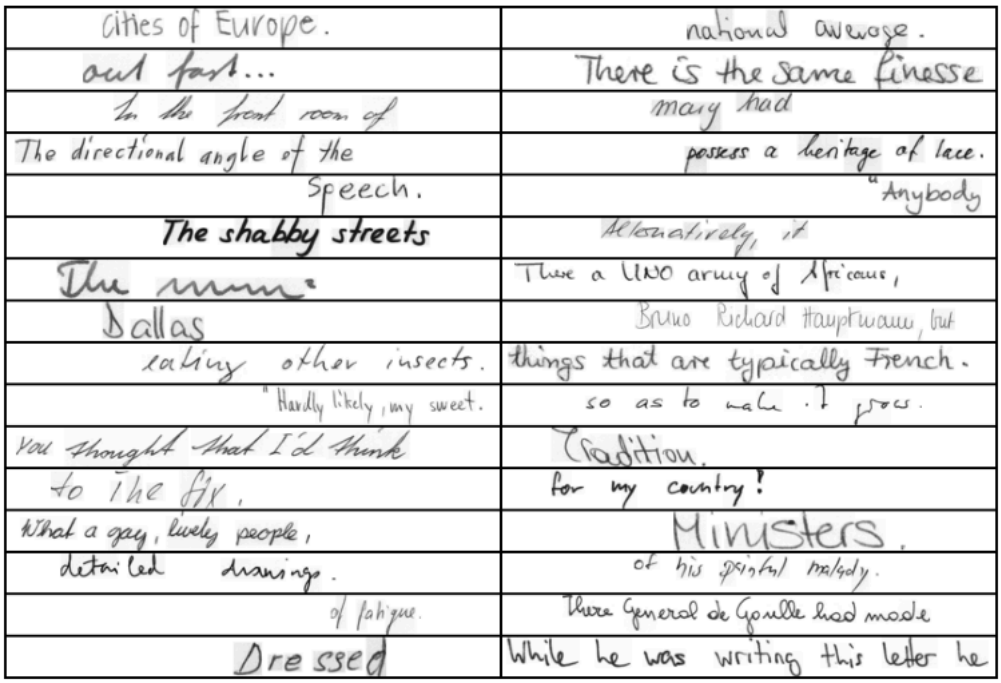
We also reserve a special `BLANK` character that CTC needs.

Note that `ImageFolder` loads RGB color images, but color is not a very useful feature for this particular problem. We will convert the images to grayscale by using the `torchvision.transforms.Grayscale` transform.

```
CHARS = "~=" + " abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789,.'-!?:;\"" #definition of characters #string
BLANK = 0
PAD = 1
CHARS_DICT = {c: i for i, c in enumerate(CHARS)} # making a dictionary: which is convert characters into numbers
TEXTLEN = 30


class SentenceDataset(torch.utils.data.Dataset):
    def __init__(self, root, transform):
        self.images = torchvision.datasets.ImageFolder(root,transform=transform)  # loads images from the given root directo
        text = []
        lengths=[]
        with open(os.path.join(root, "sentences.txt")) as f:  #opens "sentences.txt"
            for line in f:
                codes = [CHARS_DICT[c] for c in line.strip()[:TEXTLEN]]
                length = len(codes)
                codes += [PAD] * (TEXTLEN - length)
                text.append(codes)
                lengths.append(length)  #Stores the encoded sentences and their original lengths
        self.sentences = torch.tensor(text, dtype=torch.uint8)
        self.lengths = torch.tensor(lengths) #Converts the list of sentences and lengths into PyTorch tensors

    def __len__(self):
        return len(self.images)  #returns the number of samples in the dataset

    def __getitem__(self, index):
        image, _ = self.images[index]
        return image, self.sentences[index], self.lengths[index] #Retrieves the image at the given index. Returns the image,


# Image Transformation Pipeline
transform = torchvision.transforms.Compose([
    torchvision.transforms.Grayscale(num_output_channels=1), #Converts RGB images to grayscale with 1 channel
    torchvision.transforms.ToTensor(),  #Converts the image to a PyTorch tensor and scales pixel values to [0, 1]
    torchvision.transforms.Normalize(0.5, 0.5)  # Normalizes the tensor
])

#Dataset and DataLoader
train_set = SentenceDataset("handwritten/train", transform)
train_loader = torch.utils.data.DataLoader(train_set, batch_size=32, shuffle=True, drop_last=True, num_workers=2)

def decode_text(codes):
    return "".join(CHARS[c] for c in codes)  #convert a list of character indices back into a readable string

#Batch Visualization Function
def show_batch(batch):
    plt.rcParams["figure.dpi"] = 150
    g = torchvision.utils.make_grid(batch, normalize=True, value_range=(-1, 1), nrow=2 ) # Adjusts pixel values for display
    plt.imshow(g.permute(1, 2, 0), detach().cpu().numpy()) # Converts tensor shape from (C, H, W) to (H, W, C)& Prepares the
```

```
    plt.axis("off")
    plt.show()

images, sentences,lengths = next(iter(train_loader)) # retrieve one batch of data from the PyTorch DataLoader named train_lo
print(images.shape, images.dtype)
print(sentences.shape, images.dtype)
print(lengths.shape, images.dtype) # Print Shapes and Data Types

show_batch(images)   #Visualize the Image Batch
for sentence, length in zip(sentences, lengths):
    print(decode_text(sentence), length.item())

test_set = SentenceDataset("handwritten/test", transform=transform)
test_loader = torch.utils.data.DataLoader(test_set, batch_size=32, shuffle=False, drop_last=False, num_workers=2)
```

```
torch.Size([32, 1, 32, 400]) torch.float32
torch.Size([32, 30]) torch.float32
torch.Size([32]) torch.float32
```




```
cities of Europe .============ 18
national average .============ 18
out fast ... .================ 14
There is the same finesse===== 25
In the front room of========== 20
Mary had====================== 8
The directional angle of the== 28
possess a heritage of lace .== 28
speech .====================== 8
" Anybody===================== 9
The shabby streets============ 18
Alternatively , it============ 18
The num-====================== 8
There a UNO army of Africans , 30
Dallas======================== 6
Bruno Richard Hauptmann , but= 29
eating other insects .======== 22
things that are typically Fren 30
" Hardly likely , my sweet .== 28
so as to make it grow .======= 23
You thought that I 'd think=== 27
Tradition .=================== 11
to The Six .================== 12
for my country !============== 16
What a gay , lively people ,== 28
Ministers .=================== 11
detailed drawings .=========== 19
of his painful malady .======= 23
of fatigue .================== 12
There General de Gaulle had ma 30
Dressed===================== 7
While he was writing this lett 30
```

Model

CTC requires a model that assigns a class label (or a blank) to each element in an input sequence.

We will build a hybrid model: first, a convolutional network will transform the original 400×32 image into a sequence of 100 feature vectors.

Then, a bidirectional LSTM will assign the probability estimates to all elements in the sequence.

The convolutional part is a sequence if *residual blocks* computing

$$y = \mathrm{relu}(F_2(\mathrm{relu}(F_1(x))) + x)$$

where $F_{\{1,2\}}$ are the composition of a convolution and a batch normalization. The skip connection $\cdots + x$ is replaced by a 1×1 convolution when the shape of the two addends do not match. The blocks include an optional stride, and a final dropout.

```
class Residual(torch.nn.Module):
    def __init__(self, in_channels, out_channels, stride, dropout=0.2):
        super().__init__()
        self.conv1 = torch.nn.Conv2d(in_channels, out_channels, 3, stride, 1)
        self.bn1 = torch.nn.BatchNorm2d(out_channels)
        self.conv2 = torch.nn.Conv2d(out_channels, out_channels, 3, 1, 1)
        self.bn2 = torch.nn.BatchNorm2d(out_channels)
        if in_channels != out_channels or stride != 1:
            self.skip = torch.nn.Conv2d(in_channels, out_channels, 1, stride, 0)
        else:
            self.skip = torch.nn.Identity()
        self.dropout = torch.nn.Dropout2d(dropout)

    def forward(self, x):
        y = torch.nn.functional.relu(self.bn1(self.conv1(x)))
        y = torch.nn.functional.relu(self.bn2(self.conv2(y)) + self.skip(x))
        y = self.dropout(y)
        return y

res = Residual(1, 32, 1)
x = torch.zeros(1, 1, 32, 400)
y = res(x)
print(x.shape, "=>", y.shape)
```

```
torch.Size([1, 1, 32, 400]) => torch.Size([1, 32, 32, 400])
```

The full model includes a feature extractor of 9 residual blocks, the bidirectional LSTM, and a final linear layer.

```
class TextRecognizer(torch.nn.Module):
    def __init__(self, labels):
        super().__init__()
        self.feature_extractor = torch.nn.Sequential(
            Residual(1, 32, 1),
            Residual(32, 32, 2),
            Residual(32, 32, 1),
            Residual(32, 64, 2),
            Residual(64, 64, 1),
            Residual(64, 128, (2,1)),
            Residual(128, 128, 1),
            Residual(128, 128, (2, 1)),
            Residual(128, 128, (2, 1)),
        )
        self.recurrent = torch.nn.LSTM(128, 128, 1, bidirectional=True)
        self.output = torch.nn.Linear(256, labels)

    def forward(self, x):
        x = self.feature_extractor(x)
        x = x.squeeze(2)
        x = x.permute(2, 0, 1)
        x, _ = self.recurrent(x)
        x = self.output(x)
        return x
```

```
recognizer = TextRecognizer(len(CHARS))
x = torch.zeros(1, 1, 32, 400)
y = recognizer(x)
print(x.shape, "=>", y.shape)
ps = sum(p.numel() for p in recognizer.parameters())
print(ps, "parameters")
```

```
torch.Size([1, 1, 32, 400]) => torch.Size([100, 1, 74])
1613514 parameters
```

Training

```
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print("Device:", DEVICE)
```

```
LR = 1e-3

recognizer.to(DEVICE)
optimizer = torch.optim.Adam(recognizer.parameters(), lr=LR)
```

```
Device: cuda
```

Utility functions

These are used during training and evaluation.

```
def ctc_decode_sequence(seq):
    """Removes blanks and repetitions from the sequence."""
    ret = []
    prev = BLANK
    for x in list(seq) + [BLANK]:
        if prev != BLANK and prev != x:
            ret.append(prev)
        prev = x
    return ret

def ctc_decode(codes):
    """Decode a batch of sequences."""
    ret = []
    for cs in codes.T:
        ret.append(ctc_decode_sequence(cs))
    return ret

def accuracy(logits, targets):
    """Computes the accuracy of the predictions, ignoring padding."""
    ok = 0
    tot = 0
    predictions = ctc_decode(logits.argmax(2))
    for ps, ts in zip(predictions, targets):
        ok += sum(p == t for p, t in zip(ps, ts) if t != PAD)
        tot += sum(t != PAD for t in ts)
    return ok / tot

def save_model(filename):
    data = {
        "recognizer": recognizer.state_dict(),
        "optimizer": optimizer.state_dict()
    }
    torch.save(data, PROJECT_DIR + filename)

def load_model(filename):
    data = torch.load(PROJECT_DIR + filename)
    recognizer.load_state_dict(data["recognizer"])
    optimizer.load_state_dict(data["optimizer"])
```

Training loop

The main component here is the CTC loss. Pytorch includes the `CTCLoss` module implenting it (documentation). This module takes as arguments:

- the logarithm of the probability estimates, that can be computed with the `log_softmax` function (combining `log` and `softmax` would make the computation be less accurate).
- The batch of target class labels.
- The lengths of the sequences of probability estimates (100, in our case).
- The lengths of the target sequences, not counting padding.

The problem is challenging, and many epochs are required to get decent results.

```
EPOCHS = 50  # Try with 50, or more

recognizer.train()

ctc_loss = torch.nn.CTCLoss()
steps = 0
for epoch in range(EPOCHS):
    for x, y, lengths in train_loader:
        x = x.to(DEVICE)
        y = y.to(DEVICE)
        optimizer.zero_grad()
        logits = recognizer(x)
        ilengths = torch.full((logits.shape[1],), logits.shape[0])
        logp = torch.nn.functional.log_softmax(logits, 2)
```

```
        loss = ctc_loss(logp, y, ilengths, lengths)
        loss.backward()
        optimizer.step()
        steps += 1
        if steps % 100 == 0:
            acc = accuracy(logits, y)
            print(f"{steps}[{epoch}] {loss.item():.3f} {100 * acc:.3f}%")
    save_model("hw.pt")
```

```
100[0] 3.553 0.000%
200[1] 3.585 0.000%
300[2] 3.176 0.000%
400[2] 3.154 0.000%
500[3] 2.957 0.000%
600[4] 2.673 1.933%
700[4] 2.734 1.493%
800[5] 2.687 2.689%
900[6] 2.497 1.183%
1000[7] 2.465 4.045%
1100[7] 2.239 3.938%
1200[8] 2.236 4.259%
1300[9] 2.074 8.266%
1400[9] 1.890 5.288%
1500[10] 1.842 8.612%
1600[11] 1.901 9.359%
1700[12] 1.834 14.235%
1800[12] 1.627 11.238%
1900[13] 1.268 22.439%
2000[14] 1.182 31.317%
2100[14] 1.393 22.757%
2200[15] 1.081 30.164%
2300[16] 1.173 30.564%
2400[17] 1.143 34.673%
2500[17] 1.110 32.823%
2600[18] 1.047 32.342%
2700[19] 0.897 39.439%
2800[19] 1.016 27.670%
2900[20] 0.759 49.570%
3000[21] 0.905 39.586%
3100[22] 0.736 51.336%
3200[22] 0.776 48.173%
3300[23] 0.754 48.829%
3400[24] 0.706 52.496%
3500[24] 0.722 47.069%
3600[25] 0.777 40.803%
3700[26] 0.616 59.153%
3800[27] 0.486 61.014%
3900[27] 0.792 48.309%
4000[28] 0.722 42.388%
4100[29] 0.615 56.361%
4200[29] 0.612 53.388%
4300[30] 0.692 51.685%
4400[31] 0.558 51.017%
4500[32] 0.506 71.725%
4600[32] 0.521 63.506%
4700[33] 0.577 44.569%
4800[34] 0.501 52.413%
4900[34] 0.471 65.328%
5000[35] 0.770 58.172%
5100[36] 0.473 51.993%
5200[37] 0.496 61.228%
5300[37] 0.548 52.117%
5400[38] 0.374 76.442%
5500[39] 0.587 54.779%
5600[39] 0.493 68.412%
5700[40] 0.500 60.884%
5800[41] 0.337 81.579%
```

Evaluation

To evaluate the model, we need a perfomance measure. The accuracy of the prediction of single characters is not a good measure, because:

- predictions are spread over multiple elements in the sequence;
- output and target sequences may have different lengths (in case of errors).

A simple alternative is sequence level accuracy, that is, the fraction of sequences correctly identified. However, this would count equally bad sequences with one error and with many errors.

A more sophisticated error measure is the edit distance, also called [Levenshtein distance]), which counts the minimal number of editing operations (insertions, deletions, substitutions) to transform the prediction in the target sequence.

The edit distance can be effectively computed with a $O(n^2)$ dynamic programming algorithm, and is implemented in the `edit_distance.py` module.

```
load_model("hw.pt")
recognizer.eval() #set to evaluation model

tot_acc = 0
tot_dist = 0
n = 0
for x, y, lengths in test_loader:
    x = x.to(DEVICE)
    y = y.to(DEVICE)
    with torch.no_grad():
        scores = recognizer(x)
    predictions = scores.argmax(2)
    decoded = ctc_decode(predictions)
    acc = accuracy(scores, y)
    tot_acc += acc * x.shape[0]
    for ps, ds, ts, length in zip(predictions.T, decoded, y, lengths):
        dist = edit_distance(ds, ts[:length])
        tot_dist += dist
        n += 1
        if n < 50:
            txt1 = decode_text(ts)
            txt2 = decode_text(ps)
            txt3 = decode_text(ds)
            print(f"{n:3d} |{txt1}|{txt2}|{txt3}| {dist}")

print(f"Accuracy: {tot_acc * 100 / n:.1f}%")
print(f"Avg distance: {tot_dist / n:.3f}")
```

```
  1 |echo-sounders .===============|~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~e~~c~h~~o~ ~~~~-~~~~~s~o~~u~~n~~~~~d~~e~rr~
  2 |Farnese , with a snigger .====|~~~~~~~~~~~~~~~~~F~~a~~m~~~~~e~ss~~e  ,  ~w~~ii~t~h~~ ~~~~~d~  ~~~~~s~n~~ii~g~gg~ee~r .
  3 |" The jackals bay=============|"~~~~~~~~~~~~~  ~~~~T~hh~~~~e~~  ~~~~~j~~~a~~~~c~k~~  ~a~~~l~~s~  ~~~~~b~~~~a~~~~~~s~~~~
  4 |written in Greek .============|~~~~~~~~~~w~~~~~~riit~tee~~n~~  ~~~~~i~nn~ ~~~~b~~~~r~~ee~~c~~k~~~  ~~.~~~~~~~~~~~~~~~~~
  5 |It seems obvious that , though|~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~I~tt  ~~e~e~~n~~s   ~o~b~nn~o~u~s  ~t~h~o~t , thhaa~g~~h~
  6 |psychologist I am constantly==|~~~~~~~~~~~N~~s~~y~cch~~~o~~l~o~~g~uis~~t  ~I ~a~~m~~~~~  ~~~c~o~~u~~~s~~t~u~e~n~~~~t~l~
  7 |The definition of an==========|T~~~~~~~~~~~~~~~~~~~~~~~~h~~~~e~~  ~~~~~~~~d~e~~~f~~ii~n~~~ii~t~~i~~o~~n~~~ ~~~~~o~~f~
  8 |Near the end of a=============|~~~~~~~~~~~~~~~~~~~~~~~~~~N~~~ee~a~~r   ~t~h~~e  ~~e~nn~~~t  ~~o~f~  ~~~~a~~~~~~~~~~~~~
  9 |Sir R. Peel was here , I======|~~~~~~~~~~~~~~~~S~~~r~ ~~~~R~~~.. ~P~e~~e~l  ~~~~~~w~~~a~~s  ~~~~~h~~er~e~~  ,  ~~~!~~~~
 10 |for my country !==============|~~~~~~~~f~o~r ~m~~~~y  ~c~o~u~~n~t~r~yy !~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
 11 |" Werewolves " .==============|"~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~  ~W~~~~ee~r~ee~~w~~~o~l~~~v~ee~s~   .~~~
 12 |clothes she was wearing .=====|~~~~~~~~~~~~~~~~d~o~t~h~e~r  ~~~~~s~h~~ee  ~~~w~~a~rr  ~~~w~~~e~a~~r~iinn~g  .~~~~~~~~~~
 13 |Vesuvius seems to be tired ;==|~~~~V~~e~~s~~u~~~v~~ii~nn~~~s  ~~~~~~s~~e~~e~~mm~~~~s  ~~~~t~~o~  ~~~b~~~e~  ~~~~t~~i~~r
 14 |to 19 .=======================|~~~~~~~~~~~~~~~~~~~
```

- Find text lines with few and many errors. Which characteristics most influence the accuracy of predictions?
- Is the model overfitting the data?
- Does bidirectional LSTM perform better than simple LSTM?
- Is dropout useful?

Extensions

The models presented here are by no means optimal. Try to identify the best combination of parameters, also by adjusting the number of features, blocks etc.

Other possible investigations:

- Try a fully convutional approach, with a final convolution with a large (along the sequence direction) kernel, instead of the LSTM.
- Try to incorporate some knowledge about the language to fix some of the errors (perhaps even a language model). Incorporate a beam search algorithm.

For the last two points you can take inspiration from "Sequence Modeling With CTC" by Awni Hannun.

```
import torch.nn as nn

class HTROnlyLSTM(nn.Module):
    def __init__(self, num_chars, hidden_size=256, use_bidirectional=True):
        super(HTROnlyLSTM, self).__init__()
        # ... (Include your existing CNN layers here) ...

        # Sequence Modeling Layer
        self.lstm = nn.LSTM(
            input_size=512, # Assuming 512 features from CNN
            hidden_size=hidden_size,
            num_layers=2,
            bidirectional=use_bidirectional,
            batch_first=True,
            dropout=0.3
        )

        # Output Layer
        # If bidirectional, the output dimension is hidden_size * 2
        fc_input_dim = hidden_size * 2 if use_bidirectional else hidden_size
        self.fc = nn.Linear(fc_input_dim, num_chars + 1) # +1 for CTC blank token

    def forward(self, x):
        # x shape: [batch, channels, height, width]
        # ... (CNN feature extraction) ...

        # Prepare for LSTM: [batch, seq_len, features]
        x = x.permute(0, 3, 1, 2).flatten(2)

        x, _ = self.lstm(x)
        x = self.fc(x)
        return x
```

```
def train_model(model, train_loader, test_loader, epochs=50):
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
    criterion = torch.nn.CTCLoss(blank=BLANK, zero_infinity=True)
    history = {'train_loss': [], 'val_loss': []}

    for epoch in range(epochs):
        model.train()
        train_loss = 0
        for images, sentences, lengths in train_loader:
            images, sentences = images.to(device), sentences.to(device)

            optimizer.zero_grad()
            outputs = model(images) # [Seq, Batch, Class]

            # CTC specific formatting
            input_lengths = torch.full(size=(outputs.size(1),), fill_value=outputs.size(0), dtype=torch.long)
            loss = criterion(outputs.log_softmax(2), sentences, input_lengths, lengths)

            loss.backward()
            optimizer.step()
            train_loss += loss.item()

        # Validation phase
        model.eval()
        val_loss = 0
        with torch.no_grad():
            for images, sentences, lengths in test_loader:
```

```
                images, sentences = images.to(device), sentences.to(device)
                outputs = model(images)
                input_lengths = torch.full(size=(outputs.size(1),), fill_value=outputs.size(0), dtype=torch.long)
                loss = criterion(outputs.log_softmax(2), sentences, input_lengths, lengths)
                val_loss += loss.item()

        # Save history and print progress
        history['train_loss'].append(train_loss/len(train_loader))
        history['val_loss'].append(val_loss/len(test_loader))
        print(f"Epoch {epoch+1}/{epochs} - Train Loss: {history['train_loss'][-1]:.4f}, Val Loss: {history['val_loss'][-1]

    return history
```

```
import torch.nn as nn

class HTRModelDropoutAblation(nn.Module):
    def __init__(self, num_chars, use_dropout=True):
        super(HTRModelDropoutAblation, self).__init__()
        self.use_dropout = use_dropout

        # 1. Feature Extraction (CNN)
        # Assuming the standard architecture from your notebook
        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2, 2),
            nn.Dropout2d(0.25) if use_dropout else nn.Identity(), # Dropout layer

            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2, 2),
            nn.Dropout2d(0.25) if use_dropout else nn.Identity(),

            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d((8, 1), (8, 1)) # Collapse height to 1
        )

        # 2. Sequence Modeling (RNN)
        # Input size is 128 (features) * 1 (height) = 128
        self.lstm = nn.LSTM(128, 256, bidirectional=True, batch_first=True,
                            dropout=0.5 if use_dropout else 0, num_layers=2)

        # 3. Output Layer
        # 512 = 256 hidden size * 2 for bidirectional
        # We add 1 for the CTC blank token
        self.fc = nn.Linear(512, num_chars + 1)

    def forward(self, x):
        # x: [batch, 1, 32, 400]
        x = self.conv(x) # [batch, 128, 1, 100]
        x = x.squeeze(2).permute(0, 2, 1) # [batch, 100, 128]

        x, _ = self.lstm(x) # [batch, 100, 512]
        x = self.fc(x) # [batch, 100, num_chars + 1]
        return x.permute(1, 0, 2) # [seq_len, batch, num_chars + 1] for CTC loss
```

```
# Ensure device is defined
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
num_chars = len(CHARS)

# Experiment 1: With Dropout (Baseline)
print("Starting Training: With Dropout...")
model_with_dropout = HTRModelDropoutAblation(num_chars, use_dropout=True).to(device)
history_dropout = train_model(model_with_dropout, train_loader, test_loader, epochs=50)

# Experiment 2: No Dropout (Ablated)
print("\nStarting Training: No Dropout...")
model_no_dropout = HTRModelDropoutAblation(num_chars, use_dropout=False).to(device)
history_no_dropout = train_model(model_no_dropout, train_loader, test_loader, epochs=50)
```

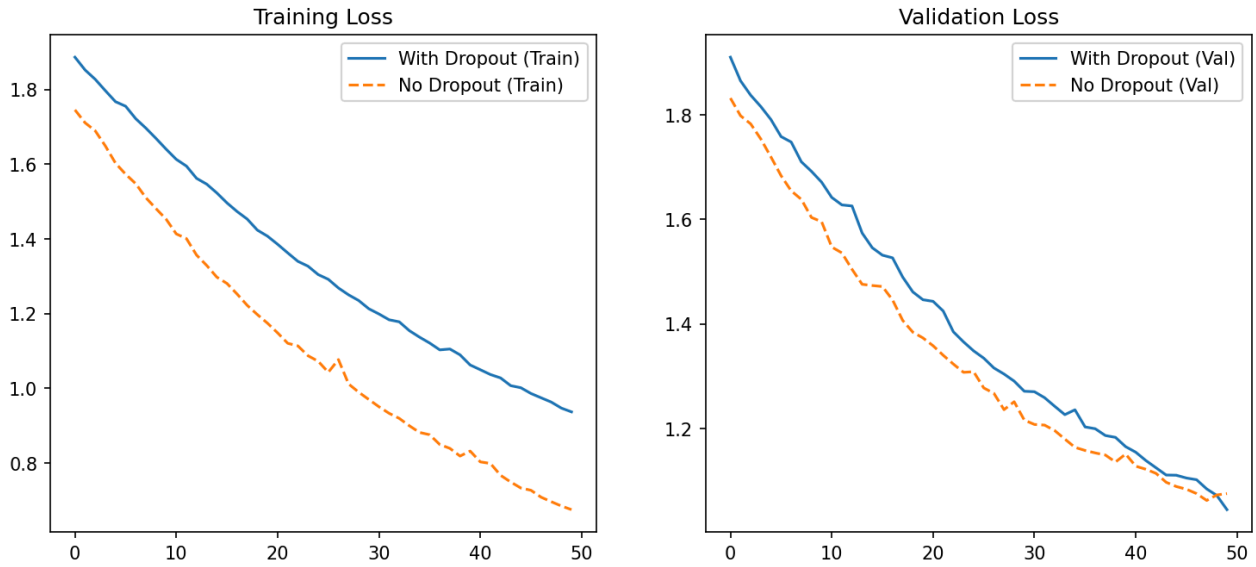
Show hidden output

```
import matplotlib.pyplot as plt

def plot_dropout_comparison(hist_drop, hist_no_drop):
    plt.figure(figsize=(12, 5))

    # Plot Training Loss
    plt.subplot(1, 2, 1)
    plt.plot(hist_drop['train_loss'], label='With Dropout (Train)')
    plt.plot(hist_no_drop['train_loss'], label='No Dropout (Train)', linestyle='--')
    plt.title('Training Loss')
    plt.legend()
```

```
    # Plot Validation Loss (Crucial for showing overfitting)
    plt.subplot(1, 2, 2)
    plt.plot(hist_drop['val_loss'], label='With Dropout (Val)')
    plt.plot(hist_no_drop['val_loss'], label='No Dropout (Val)', linestyle='--')
    plt.title('Validation Loss')
    plt.legend()

    plt.show()

# Step 2: Run Training (This takes time!)
# Ensure history_dropout and history_no_dropout are populated by train_model
history_dropout = train_model(model_with_dropout, train_loader, test_loader, epochs=50)
history_no_dropout = train_model(model_no_dropout, train_loader, test_loader, epochs=50)
```

Show hidden output

```
# Step 3: Call the function to show the plot
plot_dropout_comparison(history_dropout, history_no_dropout)
```




```
#extension
import torch.nn as nn

class FullyConvHTR(nn.Module):
    def __init__(self, num_chars):
        super(FullyConvHTR, self).__init__()
        # 1. Feature Extraction (CNN)
        self.cnn = nn.Sequential(
            nn.Conv2d(1, 64, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2, 2), # Output: [B, 64, 16, 200]
            nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2, 2), # Output: [B, 128, 8, 100]
            nn.Conv2d(128, 256, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d((8, 1), (8, 1)) # Collapse height to 1. Output: [B, 256, 1, 100]
        )

        # 2. Sequence Modeling (Large Kernel Convolution instead of LSTM)
        # We treat the width (100) as the sequence dimension.
        self.context_conv = nn.Sequential(
            nn.Conv1d(256, 512, kernel_size=11, padding=5), # Large kernel for context
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Conv1d(512, 512, kernel_size=11, padding=5),
            nn.ReLU()
        )

        # 3. Output Layer
        self.fc = nn.Conv1d(512, num_chars + 1, kernel_size=1)
```

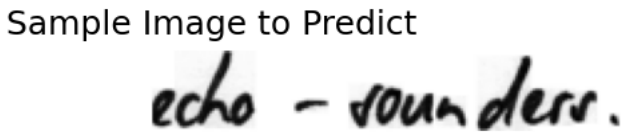
```
    def forward(self, x):
        x = self.cnn(x) # [B, 256, 1, 100]
        x = x.squeeze(2) # [B, 256, 100] (Ready for 1D conv)
        x = self.context_conv(x)
        x = self.fc(x) # [B, num_chars+1, 100]
        return x.permute(2, 0, 1) # CTC expects [Seq, Batch, Class]
```

```
import torch
import torch.nn as nn

class HTRModel(nn.Module):
    def __init__(self, num_chars, use_dropout=True):
        super(HTRModel, self).__init__()

        # 1. CNN: Extract visual features from the image
        self.cnn = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2, 2), # Reduces size
            nn.Dropout(0.2) if use_dropout else nn.Identity(),

            nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(),
            # This pool collapses the height to 1 so we get a sequence
            nn.MaxPool2d((8, 1), (8, 1))
        )

        # 2. RNN: Handle the sequence of characters
        # Input size 128 comes from the number of CNN filters
        self.lstm = nn.LSTM(128, 256, bidirectional=True, batch_first=True)

        # 3. Fully Connected: Map LSTM output to your character list
        # We add +1 for the CTC 'blank' token
        self.fc = nn.Linear(512, num_chars + 1)

    def forward(self, x):
        # x shape: [Batch, 1, Height, Width]
        x = self.cnn(x)              # Output: [Batch, 128, 1, Width/4]
        x = x.squeeze(2)             # Remove height dim: [Batch, 128, Width/4]
        x = x.permute(0, 2, 1)       # Prepare for LSTM: [Batch, Seq, Features]

        x, _ = self.lstm(x)          # Output: [Batch, Seq, 512]
        x = self.fc(x)               # Output: [Batch, Seq, Num_Chars + 1]

        # CTC Loss in PyTorch expects [Seq, Batch, Classes]
        return x.permute(1, 0, 2)
```

```
# Use the length of your predefined CHARS string
num_chars = len(CHARS)
# Initialize the model
model = HTRModel(num_chars=num_chars, use_dropout=True).to(device)

# Get a batch of data
images, sentences, lengths = next(iter(test_loader))
# Take the first image from that batch
# Shape will be [1, 32, 400] -> (Channels, Height, Width)
sample_image = images[0]
```

```
import torch

def decode_greedy(model_output, char_list):
    # 1. Get the most likely character index at each position for the first item in batch
    # model_output shape: [Seq, Batch, Class] -> arg_maxes shape: [Seq]
    arg_maxes = torch.argmax(model_output, dim=2)[:, 0]

    # 2. Collapse duplicates and remove blanks (index 0 is BLANK)
    decoded_indices = []
    previous_idx = -1

    for idx in arg_maxes:
        idx = idx.item()
        if idx != previous_idx:
            if idx != 0: # 0 is the BLANK token index
                decoded_indices.append(idx)
        previous_idx = idx

    # 3. Convert indices to actual text
    # Directly map index to char_list (no i-1)
```

```
    return "".join([char_list[i] for i in decoded_indices])


# Prepare the image: add batch dimension and move to GPU/CPU
# Assuming sample_image is [1, Height, Width]
input_tensor = sample_image.unsqueeze(0).to(device)

# Put model in eval mode to disable dropout
model.eval()

with torch.no_grad():
    # Get model predictions
    output = model(input_tensor)

# Decode using the corrected greedy approach
predicted_text = decode_greedy(output, CHARS)

print(f"Predicted: {predicted_text}")
```

```
Predicted: W
```

```
import matplotlib.pyplot as plt

# Convert tensor back to image format for plotting
plt.imshow(sample_image.squeeze(), cmap='gray')
plt.title("Sample Image to Predict")
plt.axis('off')
plt.show()

# Now run the prediction code from the previous step
predicted_text = decode_greedy(model(sample_image.unsqueeze(0).to(device)), CHARS)
```